

Ingeniería y Calidad de Software

Jake & Drosh

24-06-2025

Capítulo 3: Goals of Test Automation

El **Capítulo 3** de *xUnit Test Patterns*, titulado “**Goals of Test Automation**”, explica los **objetivos fundamentales** que justifican por qué automatizar pruebas en el desarrollo de software. No se enfoca en el “cómo”, sino en el “**para qué**”. A continuación te explico todos sus puntos clave:

1. ¿Por qué probar (y automatizar)?

- **Automatizar pruebas tiene un costo inicial alto**, pero si se hace bien, **se recupera rápidamente** con menos errores, menos pruebas manuales y menos tiempo de depuración.
 - Muchas veces, **los equipos abandonan las pruebas automatizadas cuando se complican**, lo cual es un error: allí es cuando **más se necesitan**.
 - Hay una “joroba de inversión”: al principio se invierte más, pero luego **los ahorros en calidad y tiempo la compensan**.
 - Si las pruebas se diseñan mal, el costo de mantenimiento puede crecer mucho. Por eso es clave enfocarse en **objetivos claros** desde el inicio.
-

2. Objetivos generales de la automatización de pruebas

A. Mejorar la calidad del software

- **Pruebas como especificación**: en enfoques como TDD, las pruebas se escriben antes del código y definen el comportamiento esperado. Actúan como **especificaciones ejecutables**.

- **Repelente de errores:** las pruebas automatizadas detectan errores antes de que lleguen a producción. Son una defensa activa contra regresiones.
-

B. Mejorar la comprensión del sistema

- **Pruebas como documentación:** una prueba bien escrita describe claramente qué hace una parte del sistema.
 - Las pruebas auto-verificables (**self-checking**) contienen las **aserciones que explicitan el resultado esperado**, sirviendo como guía para otros desarrolladores.
-

C. Reducir el riesgo de cambios

- **Pruebas como red de seguridad:** en sistemas sin pruebas (código heredado), hacer cambios es riesgoso. Las pruebas automatizadas permiten hacer cambios **con confianza**, sabiendo que si algo se rompe, será detectado rápidamente.
-

D. Facilitar la ejecución frecuente de pruebas

Para que las pruebas sean útiles deben ser fáciles de ejecutar. Esto requiere:

- **Pruebas completamente automatizadas:** sin pasos manuales.
- **Pruebas auto-verificables:** que puedan decir si pasaron o fallaron sin intervención.
- **Pruebas repetibles:** siempre dan el mismo resultado en cualquier entorno.
- **Pruebas independientes:** no deben depender de otras pruebas para ejecutarse.

Cuando se cumplen estos cuatro criterios, las pruebas pueden ejecutarse **frecuente y fácilmente**, incluso con un solo clic.

E. Facilitar su escritura y mantenimiento

- **Pruebas simples:** deben verificar **una sola cosa** y ser cortas.

- **Pruebas expresivas:** deben estar escritas en un lenguaje claro y cercano al dominio del negocio. Esto se puede lograr usando **métodos de utilidad** que encapsulan lógica compleja.
 - **Separación de intereses:** cada prueba debe ocuparse de un solo aspecto y no mezclar responsabilidades. Además, el código de pruebas debe estar separado del código de producción.
-

F. Reducir el esfuerzo de mantenimiento con el tiempo

- **Pruebas robustas:** deben resistir los cambios. Es decir, cuando se modifica una parte del sistema, la menor cantidad posible de pruebas debe verse afectada.
 - Esto se logra **aislando bien cada prueba**, evitando que dependan de detalles innecesarios del entorno o de otras partes del sistema.
-

Conclusión

El capítulo plantea que **automatizar pruebas no es un fin en sí mismo**, sino una herramienta para:

- Aumentar la calidad del software.
- Facilitar el cambio seguro.
- Mejorar la comprensión del sistema.
- Minimizar el costo a largo plazo.

Todo esto solo se logra **si las pruebas están bien diseñadas**, con objetivos claros y buenas prácticas desde el inicio. El resto del libro desarrollará cómo lograr esos objetivos.

Capítulo 11: Using Test Doubles

El **Capítulo 11** de *xUnit Test Patterns*, titulado “**Using Test Doubles**”, trata en profundidad **cómo usar dobles de prueba para aislar al sistema bajo prueba (SUT)** de sus dependencias, facilitando pruebas más controladas, rápidas, confiables y

mantenibles. A continuación, te explico sus contenidos clave de forma estructurada y clara:

1. Entradas y salidas indirectas

Entradas indirectas

- Son datos que el SUT **recibe desde sus dependencias** (DOC – *Depended-On Component*).
- Pueden ser:
 - Valores devueltos.
 - Parámetros modificados.
 - Excepciones lanzadas.

¿Cómo se controlan?

- Reemplazando el DOC por un **Test Stub** configurado para devolver datos pre-definidos.
- Así se puede forzar al SUT a recorrer distintas rutas de ejecución.

Salidas indirectas

- Son acciones que el SUT **envía a sus dependencias**, como llamadas a métodos o mensajes.

¿Cómo se verifican?

- De dos formas:
 1. **Verificación procedural**: se graban las llamadas al DOC y se revisan luego → se usa un **Test Spy**.
 2. **Verificación por especificación**: se define qué llamadas se esperan y se verifica automáticamente durante la ejecución → se usa un **Mock Object**.

2. Tipos de Test Doubles

Dummy Object

- Objeto “relleno”.
- Se pasa como argumento pero **no se usa en realidad**.

Test Stub

- Sustituye una dependencia para **controlar entradas indirectas**.
- Tipos:
 - **Responder**: Devuelve valores normales → camino feliz.
 - **Saboteur**: Lanza errores → para probar manejo de excepciones.
 - **Entity Chain Snipping**: Simplifica redes de objetos.

Test Spy

- Similar a un stub, pero **registra lo que ocurre** (llamadas al DOC).
- Permite verificar salidas después de ejecutar el SUT.
- Puede incluir:
 - **Retrieval Interface**: permite acceder a los registros.
 - **Self Shunt**: la misma prueba actúa como espía.

Mock Object

- Se configura con **expectativas explícitas** (método, orden, argumentos).
- Verifica automáticamente si el SUT cumple con esas expectativas.
- Falla **en el momento exacto** si no lo hace.

Fake Object

- Implementa una **versión funcional simplificada** de la dependencia.
 - Ejemplo: base de datos en memoria.
-

3. Cómo construir Test Doubles

Hechos a mano (Hand-Built)

- Escritos manualmente.
- Pueden ser:
 - **Hard-coded**: Respuestas fijas.
 - **Configurables**: Se ajustan desde la prueba.

Generados dinámicamente

- Se crean automáticamente con un *framework* como Mockito.
 - Son configurables por naturaleza.
-

4. Configuración de Test Doubles

- Algunos deben ser **configurados explícitamente**:
 - Con valores a devolver.
 - Con expectativas de llamada.

Métodos de configuración:

- **Hard-coded**: El programador define los valores fijos.
 - **Configurables**: Se ajustan en tiempo de ejecución, reduciendo duplicación y aumentando la legibilidad.
-

5. Instalación de Test Doubles

Para que el SUT use el Test Double en lugar del componente real, se lo debe “inyectar” de alguna forma:

Inyección de Dependencias

- Por constructor o método setter.
- **Forma preferida** en pruebas unitarias.

Búsqueda de Dependencias

- El SUT busca la dependencia en un servicio externo configurado por la prueba (Service Locator).

Subclase específica de prueba

- Se hereda del SUT y se sobrescriben métodos para insertar el double.

Test Hook

- Se agrega código especial en el SUT para facilitar su prueba.
 - Suele ser una **solución temporal** para código difícil de testear.
-

6. Otros usos de Test Doubles

Además de controlar entradas y verificar salidas, también sirven para:

- **Endoscopic Testing:** Examinar el comportamiento interno del SUT usando mocks como sensores.
 - **Need-Driven Development:** Definir cómo debe comportarse una dependencia *antes* de que exista, usando mocks.
 - **Acelerar configuración de fixtures:** Reemplazar estructuras complejas con dobles.
 - **Acelerar ejecución de pruebas:** Usar *fakes* en vez de sistemas reales lentos.
-

7. Advertencias importantes

- **Siempre probar con implementaciones reales** al menos una vez, para evitar confiar solo en dobles incorrectos.
 - **No modificar el SUT** solo para que sea testeable. Se debe mantener su lógica intacta.
 - **Evitar pruebas frágiles:**
 - Demasiados mocks o stubs pueden hacer que las pruebas **fallen ante cualquier cambio menor**.
 - El software queda **sobreespecificado**, dificultando la evolución del diseño.
 - **El mantenimiento es costoso** si se escriben malas pruebas (poca claridad, alta duplicación, dependencia del orden, etc.).
-

En resumen

El capítulo enseña que los **Test Doubles** permiten probar componentes en aislamiento, pero deben usarse:

- **Con criterio**, para evitar fragilidad.
- Con **patrones de diseño adecuados** (inyección de dependencias, separación de intereses).
- Para **mejorar la velocidad, control y observabilidad** de las pruebas.

El buen uso de estos patrones es clave para construir **pruebas confiables, mantenibles y útiles a largo plazo**.